BUGFIXES

BUGFIXES

Root-cause-analysed bug fixes for this project, per the Universal Mandatory Constraints (CLAUDE.md mandate #10). Each entry records the defect, root cause, affected files, the fix, and a link to the reproduction/verification evidence.

BUGFIX-0001 — `run-tests.sh` aborts after the first test under `set -e`

- **Type:** Bug (script-internal failure — Helix Constitution §11.4.1)
- **Status:** Fixed
- **Date:** 2026-06-30
- **Affected file:** `tests/run-tests.sh` (`test_result()`)

Symptom

`bash tests/run-tests.sh` printed only its banner and the first test's section header, then exited `1` (7 lines of output total) — every test after the first was silently never run, and no `TEST SUMMARY` was printed.

Reproduction (captured, run in-session before the fix)

```
$ bash -c 'set -euo pipefail; x=0; ((x++)); echo "SURVIVED,
x=$x"'; echo "exit=$?"
exit=1                      # "SURVIVED" never printed

$ bash tests/run-tests.sh 2>&1 | wc -l
7                           # aborted; exit=1
```

Root cause (FACT — not a guess, Helix Constitution §11.4.6)

`test_result()` counted with the bash post-increment idiom `(( TESTS_RUN++ ))`. The arithmetic command `(( expr ))` returns exit status **1** when `expr` evaluates to **0**. `TESTS_RUN++` is a *post*-increment, so its value is the value *before* incrementing — `0` on the very first call — making `(( TESTS_RUN++ ))` return status 1. The script runs under `set -euo pipefail` (line 7), so that non-zero status aborted the entire suite at the first `test_result` call, before any results or the summary could print. The same trap applied to the first `PASS` (`(( TESTS_PASSED++ ))`) and first `FAIL` (`(( TESTS_FAILED++ ))`).

This was a **pre-existing** latent defect (the `(( ... ++ ))` idiom was original); it surfaced while wiring the constitution-inheritance pre-flight gate in as the first test.

Fix (at source, Helix Constitution §11.4.1)

Replaced the three post-increment arithmetic commands with the assignment form, which always returns status 0 and is immune to the `set -e` trap:

```
- ((TESTS_RUN++))
+ TESTS_RUN=$((TESTS_RUN + 1))
- ((TESTS_PASSED++))
+ TESTS_PASSED=$((TESTS_PASSED + 1))
- ((TESTS_FAILED++))
+ TESTS_FAILED=$((TESTS_FAILED + 1))
```

Verification (captured, run in-session after the fix)

```
$ bash -c 'set -euo pipefail; x=0; x=$((x+1)); echo "SURVIVED,
x=$x"'; echo "exit=$?"
SURVIVED, x=1
exit=0

$ bash tests/run-tests.sh 2>&1 | wc -l
28                          # suite now runs to completion (was 7)
# first line of results:
✓ PASS: Constitution inheritance gate (§11.4.35)
```

The suite now executes every test and prints its summary. (Its overall exit remains non-zero in an environment without the running proxy services — those are real-infrastructure tests that correctly FAIL/skip when the live System is absent, Helix Constitution §11.4.11 — that is a separate, pre-existing, infrastructure-dependent condition, not this defect.)

> **Cross-reference (§11.4.138):** the “proxy services not running” condition noted above was treated as merely environmental. It was not — BUGFIX-0002 below is the *reason* the proxy would not stay up under rootless Podman. Driving the real data path (not just “tests pass”) surfaced it.

BUGFIX-0002 — squid crash-loops under rootless Podman (log dir not writable) → proxy never serves

- **Type:** Bug (product defect — end-user-visible: the proxy does not work)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `lib/container-runtime.sh` (`create_directories()`), `start` (`init_cache()`)
- **Regression guard:** `tests/regression/log_dir_writable_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)
- **Workable item:** #38 (the live-usability proof stream)

Symptom

A freshly-booted proxy (`./start --no-vpn`, rootless Podman) returned `http_code=000` for every request through it. `proxy-squid` was not `(healthy)` — it was restarting in a loop, never accepting a connection, so **no feature of the proxy worked for the end user** — the exact “tests pass but the feature can’t be used” failure mode §11.4 forbids.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Live boot of the real orchestrator, then curl through the proxy:
$ curl -s -o /dev/null -w '%{http_code}' -x http://127.0.0.1:53128
http://example.com/
000

# squid's own log (read from inside the container) showed the
FATAL:
$ podman logs proxy-squid | tail
FATAL: Cannot open '/var/log/squid/access.log' for writing.
       The parent directory must be writeable by the user 'proxy',
       which is the cache_effective_user set in squid.conf.

# Unit reproduction of the orchestrator code path (pre-fix
replica):
$ RED_MODE=1 tests/regression/log_dir_writable_test.sh
```

```
[PASS] BUGFIX#38 log-dir-writable (RED_MODE=1): RED reproduced:
       pre-fix LOG_DIR mode=755 is NOT world-writable
```

Root cause (FACT — not a guess, §11.4.6)

Under **rootless** Podman the invoking host uid (1000) maps to the container's `root`, but every other container uid is remapped through `/etc/subuid` into a high host range. `squid.conf` sets `cache_effective_user proxy`, so squid drops privileges to its non-root `proxy` user — which, on the host, is a high subuid that owns **none** of the host-created bind-mount dirs. The host `./logs` directory was created mode `0755` (owner-write only), so the container `proxy` user had only world `r-x` — no write — and squid `FATAL`ed opening `access.log`, then crash-looped. The orchestrator already `chmod 777`-ed the **cache** bind-mount (`init_cache`) for this exact reason, but the **log** bind-mount was missed — a single missing site of an already-known remedy.

Fix (at source, §11.4.1 / §11.4.108 SOURCE layer)

Make the squid log bind-mount world-writable in the orchestrator's dir-init, mirroring the existing cache `chmod 777`. Applied at **both** self-sufficient sites so it holds regardless of call order:

```
# lib/container-runtime.sh  create_directories()
  mkdir -p "$LOG_DIR"
+ chmod 777 "$LOG_DIR"

# start  init_cache()
+ mkdir -p "$LOG_DIR"
+ chmod 777 "$LOG_DIR"
```

Verification (captured, run in-session after the fix)

Unit — regression guard, §11.4.115 polarity (RED reproduces, GREEN proves):

```
$ RED_MODE=1 tests/regression/log_dir_writable_test.sh   # pre-fix
replica
[PASS] RED reproduced: LOG_DIR mode=755 NOT world-writable
$ tests/regression/log_dir_writable_test.sh              # real
fixed code
[PASS] GREEN: create_directories() made LOG_DIR mode=777 world-
writable
```

§1.1 paired mutation (guard is not a tautology): stripping the `chmod 777 "$LOG_DIR"` from `create_directories()` made the GREEN guard **FAIL** (`REGRESSION: … mode=755 NOT world-writable`, exit 1); the

lib was restored byte-identical (md5 `0128a96b6d467c2da5b7cef8a808e563` before == after) and the guard PASSed again.

Live data-plane (§11.4.108 RUNTIME + USER-VISIBLE): after the fix the booted proxy serves real HTTP through it:

```
$ curl -x http://127.0.0.1:53128 http://example.com/ ->
http_code=200
Via: 1.1 proxy-squid (squid/6.13)
# squid access.log (the file that FATAL'd pre-fix) — real request
logged:
1782858066.886  189 10.89.1.249 TCP_MISS/200 951 GET http://
example.com/ - HIER_DIRECT/104.20.23.154 text/html
$ podman ps  ->  proxy-squid  Up (healthy)
```

Evidence: `qa-results/regression/bugfix38/`.

BUGFIX-0003 — `tests/run-tests.sh` aborts mid-suite on a no-message FAIL (`set -e`); ~29 of 39 tests silently never ran

- **Type:** Bug (script-internal failure — §11.4.1; sibling of BUGFIX-0001)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected file:** `tests/run-tests.sh` (`test_result()`)
- **Regression guard:** `tests/regression/test_result_returns_zero_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)

Symptom

`bash tests/run-tests.sh` printed only ~10 results then stopped — no `TEST SUMMARY`, exit 1. The Directory / Config / Compose / Runtime / Port / Cache / VPN / Service-startup groups after the first FAIL never ran, so the suite *appeared* to run while actually exercising under a third of its tests and hiding real failures.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Extract the pre-fix test_result and call it with a no-message
FAIL under set -e:
$ bash -c 'set -euo pipefail; <test_result>; test_result "x"
"FAIL"; echo SURVIVED'
✗ FAIL: x
```

```
# exit=1 — "SURVIVED" never printed (the function returned 1 and
aborted)

$ RED_MODE=1 tests/regression/test_result_returns_zero_test.sh
[PASS] RED reproduced: pre-fix test_result aborts under set -e on
a no-message FAIL
```

Root cause (FACT — not a guess, §11.4.6)

`test_result()`'s FAIL branch ended on `[[ -n "$message" ]] && echo -e "  → $message"`. When `message` is empty (every `test_result "..." "FAIL"` call with no third argument), the `[[ -n "$message" ]]` test is false, the `&&` list short-circuits, and the list — the **last command of the function** — returns exit status **1**. When such a `test_result` is the last command executed in a test function (e.g. the final `"Upstreams"` iteration of `test_directories`'s `for` loop, where the dir is absent → FAIL), that function returns 1, and under `set -euo pipefail` `main()` aborts immediately — every later test group and the summary never run. This is the same §11.4.1 class as BUGFIX-0001 (a reporting helper leaking a non-zero status into control flow), a *different* site (`test_result`'s tail, not the `(( ++ ))` counter), so BUGFIX-0001's fix did not cover it.

Fix (at source, §11.4.1)

`test_result` is a reporting helper; its exit status must never gate control flow. Append an explicit `return 0`:

```
         [[ -n "$message" ]] && echo -e "  ${YELLOW}→ $message$
        {NC}"
    fi
+
+    return 0
 }
```

Verification (captured, run in-session after the fix)

```
# §11.4.115 GREEN (real fixed test_result survives a no-message
FAIL):
$ tests/regression/test_result_returns_zero_test.sh
[PASS] GREEN: real test_result returns 0 on a no-message FAIL

# Full suite now runs to completion (was 32 lines, aborted):
$ bash tests/run-tests.sh   # 82 lines, reaches TEST SUMMARY
Tests Run:    41
Tests Passed: 34
Tests Failed: 7
```

§1.1 paired mutation (guard is not a tautology): deleting the `return 0` re-introduced the abort → the GREEN guard **FAILed** (`REGRESSION: test_result aborts the suite …`, exit 1); `tests/run-tests.sh` was restored byte-identical (md5 `7c2bab18c4566d081b1c8aa7a9a412e0` before == after) and the guard PASSed again.

> **Follow-up (separate, tracked):** with the abort fixed, the suite now honestly reports **7 pre-existing FAILs** (e.g. `Directory Upstreams`) that the abort had been masking. These are real existing-system findings to triage — not regressions from this fix, but newly *visible* because the suite finally runs to completion.

Evidence: `qa-results/regression/bugfix0003/`.

BUGFIX-0004 — `run-tests.sh` reports FAILURE on a HEALTHY serving proxy (§11.4.1 false-FAIL); skips recorded as PASS

- **Type:** Bug (anti-bluff — §11.4.1 false-FAIL + §11.4.3 PASS-by-default-for-skip)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected file:** `tests/run-tests.sh` (`test_result`, `test_environment`, `test_directories`, `test_container_runtime`, `test_ports` + new `port_verdict`/`_ports_check_one`, `test_vpn`, `test_service_startup`, `print_summary`)
- **Regression guard:** `tests/regression/port_topology_aware_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)
- **Bluff audit:** `docs/research/existing_test_bluffs_audit/README.md` (B7)
- **Workable item:** #47 (P8)

Symptom

Once BUGFIX-0003 let the suite run to completion, `bash tests/run-tests.sh` reported `Tests Run: 41, Passed: 34, Failed: 7` and exited **1 against a perfectly healthy, serving proxy** (proxy-squid + proxy-dante `Up (healthy)`). A suite that declares FAILURE on a working System is a §11.4.1 false-FAIL — as misleading as a false-PASS.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Against the running, healthy proxy:
$ bash tests/run-tests.sh | tail -4
Tests Run:    41
Tests Passed: 34
Tests Failed: 7         # exit 1
# The 7 FAILs: .env file exists, HTTP_PROXY_PORT set, Directory
Upstreams,
# Docker installed, Port 53128 available, Port 51080 available,
Port 58080 available

$ RED_MODE=1 tests/regression/port_topology_aware_test.sh
[PASS] RED reproduced: pre-fix logic classifies a healthy serving
proxy port
       (owner up + listening) as FAIL
```

Root cause (FACT — not a guess, §11.4.6)

Three independent bluff classes, all FACT-confirmed against the live host:

1. **Port-semantics inversion (§11.4.1).** `test_ports` reported any port that was IN USE as `FAIL` ("Port in use"). The running proxy's own listening ports (squid `53128`, dante `51080`) are *in use by their own healthy service* — the serving state — so the suite FAILed on health.
2. **Inverted runtime check (§11.4.161).** `test_container_runtime` asserted `Docker installed` and FAILed on its absence. The project MANDATES rootless Podman and FORBIDS Docker as a workflow — the check tested for the forbidden runtime and failed on the compliant state.
3. **Config-absent-as-FAIL (§11.4.3).** `.env` (gitignored, §11.4.10/ §11.4.30) and `Upstreams/` absence FAILed, though their absence is the *expected* fresh-checkout topology. And (B7) `test_result` had no SKIP state, so `test_vpn`/`test_service_startup` skipped paths were recorded as PASS, inflating `TESTS_PASSED` (§11.4.3 PASS-by-default).

Fix (at source, §11.4.1 / §11.4.3 / §11.4.161)

- **3-state `test_result`** — add a `SKIP` branch + `TESTS_SKIPPED` counter + `Tests Skipped:` summary line; the suite exit gates on `TESTS_FAILED -eq 0` only (skips are neither pass nor fail). BUGFIX-0001 assignment-form counters + BUGFIX-0003 `return 0` preserved.
- **§11.4.3 topology-aware ports** — a *pure* `port_verdict(owner_serving, listening)` truth-table (serving→PASS, owner-up-but-not-listening→FAIL, pre-start-

free→PASS, foreign-process-holds-it→SKIP); `_ports_check_one` resolves `owner_serving` from real `podman ps`/`podman port` and `listening` from real `ss` (no data-plane probe).
- **§11.4.161** — assert the mandated `podman`; Docker informational only, absence is `SKIP` not FAIL.
- **§11.4.3 SKIP** for absent gitignored `.env`/`HTTP_PROXY_PORT` (validate the tracked `.env.example` template instead) and absent `upstreams//Upstreams/`.
- **B7** — `test_vpn`/`test_service_startup` disabled paths emit `SKIP`.

Verification (captured, run in-session after the fix)

```
# Full suite vs the healthy running proxy — zero failures, honest
skips:
$ bash tests/run-tests.sh | tail -5
Tests Run:     43
Tests Passed:  37
Tests Skipped: 6
Tests Failed:  0          # exit 0 — "All tests passed (6 skipped
…)"

# §11.4.115 polarity guard (tests the REAL pure port_verdict, not
a copy):
$ RED_MODE=1 tests/regression/port_topology_aware_test.sh   #
reproduces
[PASS] RED reproduced: healthy serving port classified FAIL
$ tests/regression/port_topology_aware_test.sh              #
GREEN
[PASS] GREEN: HEALTHY=PASS NOTSERVING=FAIL PRESTART_FREE=PASS
PRESTART_BUSY=SKIP
```

§1.1 paired mutation (guard is not a tautology): flipping `port_verdict`'s PASS/FAIL branch made the GREEN guard **FAIL** (`REGRESSION: … HEALTHY=FAIL …`, exit 1) with `bash -n` still clean (assertion, not parse error); `tests/run-tests.sh` restored byte-identical (md5 `2f4d3e4bf33cd391036cee595839d439`) and the guard PASSed again. The guard is wired into `test_regression_guards()` (GREEN + RED self-check); the 5 pre-existing guards still PASS.

> **Honest note (§11.4.6 / §11.4.174):** port `58080` (the control-API port) is currently held by a **non-project** host process — no proxy container publishes it — so the suite classifies it `SKIP` (readiness not assertable), never a false-FAIL blaming the proxy. The foreign process was **not** touched (shared-host ownership). This is a real condition to free before P10 deploys the control-API there.

Evidence: `qa-results/p8/`, `qa-results/regression/portstopology/`.

BUGFIX-0005 — `final-verify.sh` + `verify-proxy.sh` greened a NO-VPN config (false-VPN-routing §15 bluff) + aborted under `set -e`

- **Type:** Bug (anti-bluff — false-VPN-routing §15 + §11.4.1 script-abort)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `tests/final-verify.sh` (audit B5), `tests/verify-proxy.sh` (audit B6)
- **Bluff audit:** `docs/research/existing_test_bluffs_audit/README.md` (B5, B6)
- **Future guard:** the standing `assert_egress_ip` invocation (live RED/GREEN at P10, §11.4.135) — SKIPs honestly until a real tunnel + `VPN_EXIT_IP` exists.
- **Workable item:** #49 (P8b)

Symptom & root cause (FACT — §11.4.6)

Both scripts declared `VPN routing verified` **PASS** when the egress IP seen THROUGH the proxy EQUALS the host's real direct IP (`host_ip == proxy_ip`). That equality is PROOF traffic was **NOT** routed through any VPN — both paths exit the same address — so the test greened the exact no-VPN configuration it claimed to forbid (the §15 bluff; `verify-proxy.sh:46`'s comment literally encoded the wrong invariant "proxy uses same IP as host"). Neither compared against an EXPECTED tunnel-exit IP. Additionally both `test_pass`/`test_fail` used `((PASS++))`/`((FAIL++))`, which returns exit 1 when the counter is 0 and aborts the script under `set -euo pipefail` (the §11.4.1 class of BUGFIX-0001/B4) — so the VPN block was unreachable until that was fixed too.

Fix (at source)

Remove the egress==host PASS entirely; assert the real data-plane contract via `evidence.sh`: egress through the proxy must equal the EXPECTED tunnel exit AND differ from the host IP; absent a configured `VPN_EXIT_IP` (no tunnel yet) emit an honest §11.4.3 SKIP (`operator_attended`) — never a fabricated PASS. `((PASS++))`/`((FAIL++))` → assignment form (`PASS=$((PASS + 1))`), matching the run-tests.sh §11.4.1 fix.

```
- [[ "$host_ip" == "$proxy_ip" && "$host_ip" != "unknown" ]] &&
        test_pass "VPN routing verified"
+ . "$SCRIPT_DIR/lib/evidence.sh"
+ if [[ -n "${VPN_EXIT_IP:-}" ]]; then
+     assert_egress_ip "http://localhost:${HTTP_PROXY_PORT}"
        "$VPN_EXIT_IP" "$host_ip" \
```

```
+         && test_pass "VPN routing (egress=$VPN_EXIT_IP, !=
        host)" || test_fail "..."
+ else
+     ab_skip_with_reason "VPN routing (egress == expected tunnel
        exit, != host)" "operator_attended"
+ fi
```

Verification (captured, run in-session after the fix)

```
# Both run to completion vs the running proxy — connectivity PASS,
VPN SKIP:
$ bash tests/final-verify.sh   -> Passed: 4 Failed: 0, exit 0
  SKIP: VPN routing (egress == expected tunnel exit, != host)
[reason: operator_attended]
$ bash tests/verify-proxy.sh   -> Passed: 4 Failed: 0, exit 0

# Inverse anti-bluff proof (the §15 condition the OLD code
GREENed): set
# VPN_EXIT_IP to the host's real IP (no tunnel => egress == host)
— new code REDs:
$ VPN_EXIT_IP=<host real ip> bash tests/final-verify.sh
  FAIL: assert_egress_ip [reason: egress IP <host> == host real IP
— traffic NOT routed via VPN (§15 bluff)]
  Failed: 1, exit 1
```

The live VPN-routing GREEN (egress == real tunnel exit, != host) requires a real tunnel + `VPN_EXIT_IP` and is deferred to P10; until then the corrected assertion SKIPs honestly. `bash -n` clean both files. Reviewed independently by the conductor (§11.4.142): diffs, both runs, the inverse proof, parse — all verified against the real tree before commit.

Evidence: `qa-results/p8b/`.